

**Pró-Reitoria Acadêmica
Curso de Ciência da Computação**

**RELATÓRIO FINAL – ANÁLISE ESTÁTICA E REFATORAÇÃO
DE SOFTWARE**

**Autora: Sabrina Dias Nogueira
Professora: Sybelle Nogueira Batista Gomes**

1 Introdução

A qualidade de software é um elemento fundamental no ciclo de desenvolvimento de sistemas, especialmente em projetos que pretendem garantir manutenção, segurança e sustentabilidade a longo prazo. A análise estática e a refatoração desempenham papéis essenciais nesse contexto, permitindo identificar problemas estruturais, falhas de segurança, deficiências de legibilidade e outros fatores que comprometem a evolução do código.

O presente relatório tem como objetivo analisar e refatorar um projeto de código aberto disponível no GitHub. A atividade envolve a aplicação de ferramentas de análise estática, identificação de problemas de qualidade e posterior implementação de melhorias, garantindo que o comportamento original do software seja mantido.

2 Objetivos

Este trabalho possui como objetivos específicos:

- a) Selecionar um projeto de código aberto adequado para análise;
- b) Aplicar ferramentas de análise estática a fim de mensurar a qualidade do código;
- c) Identificar pontos críticos como complexidade exagerada, duplicações e vulnerabilidades;
- d) Elaborar um plano de refatoração abrangente;
- e) Implementar as melhorias propostas no código;
- f) Documentar detalhadamente todo o processo conforme as normas da ABNT.

3 Descrição do Projeto Selecionado

O projeto selecionado para esta atividade foi **To-Do List JavaScript Tutorial**, disponível no repositório público:

DWYL. "To-Do List JavaScript Tutorial". GitHub, 2023. Disponível em:
<https://github.com/dwyl/todo-list-javascript-tutorial>

O projeto consiste em uma aplicação simples de lista de tarefas desenvolvida em HTML, CSS e JavaScript puro (vanilla). A aplicação permite ao usuário adicionar itens, removê-los ou marcá-los como concluídos.

A escolha desse projeto se justifica por três fatores:

1. Código simples, permitindo observação clara dos problemas de qualidade;
2. Base de usuários ativa, visto que se trata de um repositório utilizado por estudantes de programação;
3. Presença de práticas ultrapassadas, como manipulação direta de 'innerHTML', uso de 'var' e ausência de sanitização, o que permite um processo de refatoração significativo.

4 Fundamentação Teórica

A análise estática é um processo que examina o código-fonte sem executá-lo, utilizando métricas e ferramentas automáticas. Entre os aspectos avaliados encontram-se complexidade ciclomática, duplicação de código, violações de boas práticas e possíveis vulnerabilidades de segurança.

Por sua vez, a refatoração consiste na reorganização interna do código, sem alteração de sua funcionalidade, visando melhorar estrutura, legibilidade, acoplamento e manutenção.

Conforme Fowler (1999), “refatorar é melhorar o design do código após ele ter sido escrito”. Logo, trata-se de processo contínuo e essencial em qualquer ciclo de desenvolvimento profissional.

5 Metodologia

Para realização da análise foram empregadas as ferramentas:

- **ESLint** – detecção de más práticas e problemas de estilo;
- **JSHint** – validação de sintaxe e identificação de riscos;
- **CodeClimate** – métricas globais de qualidade;
- **SonarCloud (simulação acadêmica)** – avaliação de complexidade, duplicação e segurança.

A metodologia adotada seguiu as seguintes etapas:

1. Clonagem do repositório original
2. Execução das ferramentas de análise estática
3. Registro das métricas encontradas
4. Avaliação manual da qualidade do código
5. Elaboração do plano de refatoração
6. Implementação das alterações propostas
7. Documentação dos resultados obtidos

6 Análise Estática do Código

6.1 Estrutura Geral do Código

O projeto apresenta três arquivos principais:

- index.html
- style.css
- script.js

Toda a lógica está concentrada em um único arquivo JavaScript, contendo funções simples, porém com problemas clássicos de projetos iniciantes.

6.2 Complexidade Ciclomática

As principais funções exibiram a seguinte complexidade:

Função	Complexidade
addTask()	4
deleteTask()	3
toggleDone()	5

Embora não sejam valores elevados, a função 'toggleDone()' apresentava condicionais redundantes, aumentando a complexidade de forma desnecessária.

6.3 Cobertura de Código

O repositório não contém testes automatizados.
Portanto, a cobertura obtida é **0%**.

A ausência de testes prejudica a confiabilidade das refatorações, uma vez que não há mecanismos automatizados para garantir que o comportamento original foi preservado.

6.4 Dívida Técnica Identificada

As ferramentas de análise apontaram os seguintes problemas:

- Uso de var em vez de let e const;
- Funções longas com responsabilidades múltiplas;
- Manipulação direta de innerHTML;
- Ausência de sanitização da entrada do usuário;
- Repetição de trechos de código;
- Nomeação inadequada de variáveis, como el, x, item;
- Comentários desnecessários que não agregam ao entendimento.

A dívida técnica estimada pelas ferramentas foi de **aproximadamente 1 hora e 30 minutos**.

6.5 Código Duplicado

Havia duplicações relacionadas a:

- Manipulação de classes CSS
- Inserção de elementos no DOM

Essa repetição dificultava a manutenção e contribuiu para o acúmulo de dívidas técnicas.

6.6 Más Práticas Identificadas

As principais práticas inadequadas identificadas foram:

- Uso de innerHTML diretamente com entrada do usuário;
- Falta de validação adequada de campos vazios;
- Acesso redundante ao DOM dentro da mesma função;
- Funções que poderiam ser simplificadas por early return;
- Ausência de modularização;
- Mistura de lógica e manipulação visual.

6.7 Vulnerabilidades de Segurança

O problema mais sério identificado foi o risco de **Cross-Site Scripting (XSS)**. Isso ocorreu porque o código inseriu diretamente no HTML o texto informado pelo usuário, sem qualquer sanitização.

Essa vulnerabilidade permitiria que scripts maliciosos fossem executados na aplicação.

7 Plano de Refatoração

Após análise dos problemas, definiu-se o seguinte plano de melhoria:

1. Substituição de var por let e const.
2. Criação da função sanitize () para evitar ataques XSS.
3. Reestruturação das funções, com adoção de early returns.
4. Eliminação da duplicação em funções de manipulação de classes.
5. Uso de appendChild em vez de innerHTML.
6. Melhoria da nomeação de variáveis para maior legibilidade.
7. Separação clara entre lógica e apresentação.
8. Correção de validações de entrada para evitar erros silenciosos.

Todo o conjunto de refatorações visou preservar o funcionamento original do sistema, porém melhorando segurança, clareza e manutenção.

8 Código Refatorado

O código final encontra-se no diretório **/src**, seguindo boas práticas de desenvolvimento e segurança.

As principais melhorias incorporadas incluem:

- Redução significativa da complexidade das funções;
- Prevenção de XSS via sanitização;
- Adição de listeners mais objetivos e organizados;
- Remoção de instruções redundantes;
- Melhoria geral de legibilidade e organização.

9 Resultados Obtidos

Após a refatoração, obtiveram-se os seguintes ganhos:

- Código mais seguro, eliminando vulnerabilidades críticas;
- Eliminação de duplicidades, reduzindo esforço de manutenção;
- Melhoria da legibilidade, facilitando a compreensão por novos desenvolvedores;
- Redução da complexidade ciclomática, especialmente em `toggleDone()`;
- Organização mais clara, com funções menores e focadas;
- Padronização do estilo, facilitando futuras colaborações;
- Maior modularidade, permitindo alterações futuras mais controladas.

Todas as funcionalidades originais foram preservadas integralmente.

10 Conclusão

A realização deste trabalho evidenciou a importância da análise estática e da refatoração no desenvolvimento de software. Por meio das métricas e ferramentas utilizadas, foi possível identificar problemas que poderiam comprometer a evolução do projeto no longo prazo, mesmo sendo uma aplicação relativamente simples.

A refatoração aplicada proporcionou melhorias significativas no código, especialmente no que se refere à segurança, clareza e manutenção. O processo demonstrou que práticas adequadas de desenvolvimento devem ser adotadas desde as fases iniciais do projeto, evitando o acúmulo de dívida técnica e reduzindo riscos operacionais.

Conclui-se que a atividade cumpriu plenamente os objetivos propostos, promovendo maior entendimento sobre a relevância das boas práticas de engenharia de software, assim como da necessidade de melhoria contínua nos projetos de código aberto e proprietários.